



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO 2- ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

1. Signed 8-bit Addition and Subtraction using 2's Complement Representation

Given Data

A processor performs arithmetic operations using 8-bit signed 2's complement representation.

Numbers:

- First number = +45
- Second number = -23

Binary Representation

Representation of +45

Decimal 45 in binary: $45 = 00101101_2$

Therefore,

$$+45 = 00101101$$

Representation of -23

First convert +23 into binary: $23 = 00010111_2$

Find 2's complement:

1's complement: 11101000

$$\text{Add: } 11101000 + 1 = 11101001$$

Therefore,

$$\mathbf{-23 = 11101001}$$

Addition Operation: (+45) + (-23)

The processor performs binary addition:

$$00101101 \quad (+45)$$

+ 11101001 (-23)

00010110

The carry generated beyond 8 bits is ignored.

$$00010110_2 = +22$$

Therefore, **(+45) + (-23) = +22**

Subtraction Operation: (+45) - (-23)

Subtracting a negative number is equivalent to addition: $45 - (-23) = 45 + 23$

Binary addition:

00101101 (+45)

+ 00010111 (+23)

01000100

The result is:

$$01000100_2 = +68$$

Therefore, **(+45) - (-23) = +68**

Overflow Detection

In signed 8-bit representation, the range is:

$$-128 \text{ to } +127$$

Overflow occurs when:

- Addition of two positive numbers produces a negative result.
- Addition of two negative numbers produces a positive result.

In these operations, one operand is positive and the other is negative, so overflow cannot occur.

Overflow Status: No overflow detected

The processor detects overflow by comparing:

- Carry into the Most Significant Bit (MSB)
- Carry out of the Most Significant Bit (MSB)

If both carries are different, overflow occurs.

2. Carry Look-Ahead Adder (CLA) Analysis

A high-performance processor replaces a Ripple Carry Adder with a 4-bit Carry Look-Ahead Adder (CLA) to reduce delay.

Generate and Propagate Signals

For each bit position:

Generate signal:

$$G_i = A_i \times B_i$$

Propagate signal:

$$P_i = A_i \oplus B_i$$

Where:

- Generate (G) indicates that a carry will be produced.
- Propagate (P) indicates that an input carry will be passed to the next stage.

Carry Computation

The CLA calculates carries in advance using:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

These equations allow all carries to be generated simultaneously.

Comparison with Ripple Carry Adder

Ripple Carry Adder	Carry Look-Ahead Adder
Carry moves sequentially from one stage to another	Carry is calculated in advance
More propagation delay	Less delay
Simple hardware design	Complex hardware design
Slower for large bit operations	Faster for large bit operations

Reason for Improved Speed

In Ripple Carry Adders, each full adder waits for the previous carry output.

In CLA, carry values are calculated parallelly using generate and propagate signals.

Therefore, CLA significantly reduces computation time and improves processor performance.

3. Booth's Algorithm for Signed Multiplication ($-6 \times +5$)

Booth's algorithm is used for efficient multiplication of signed binary numbers.

Given Values

Multiplicand: $-6 = 1010_2$

Multiplier: $+5 = 0101_2$

Initial values:

- Accumulator (A) = 0000
- Multiplier Register (Q) = 0101
- Booth bit (Q-1) = 0

Working of Booth's Algorithm

The pair of bits Q0 and Q-1 determines the operation.

Q0 Q-1	Operation
00	No operation
01	Add multiplicand
10	Subtract multiplicand
11	No operation

After every operation, an arithmetic right shift is performed.

Step 1

$$Q_0Q_{-1} = 10$$

Operation:

Subtract multiplicand from accumulator.

Then perform arithmetic right shift.

Step 2

$$Q_0Q_{-1} = 11$$

No addition or subtraction is performed.

Arithmetic shift is applied.

Step 3

$$Q_0Q_{-1} = 01$$

Add multiplicand to accumulator.

Arithmetic shift is applied.

Final Result

After completing all iterations:

$$Product = 11100010_2$$

Converting to decimal: $11100010 = -30$

Therefore:

$$-6 \times 5 = -30$$

Advantages of Booth's Algorithm

- Handles positive and negative numbers efficiently.
- Reduces the number of addition operations.
- Processes consecutive 1s in multiplier efficiently.
- Used in modern processor multiplication units.

4. Binary Division using Restoring and Non-Restoring Division

Example

Perform: $13 \div 3$

Binary values:

$$13 = 1101_2$$

$$3 = 0011_2$$

Expected result:

$$\text{Quotient} = 0100_2$$

$$\text{Remainder} = 0001_2$$

Restoring Division Algorithm

Steps:

1. Shift the partial remainder left.
2. Subtract divisor from remainder.
3. If the result is negative:
 - Restore the previous remainder.

- Set quotient bit as 0.
4. If the result is positive:
- Keep the result.
 - Set quotient bit as 1.

Final output:

Quotient = 4

Remainder = 1

Non-Restoring Division Algorithm

Steps:

1. Shift remainder left.
2. If remainder is positive, subtract divisor.
3. If remainder is negative, add divisor.
4. Generate quotient bits based on the result.

Unlike restoring division, it does not immediately restore the remainder.

Final output:

Quotient = 4

Remainder = 1

Comparison

Restoring Division	Non-Restoring Division
Requires restoring operation	No restoring operation
More arithmetic operations	Fewer arithmetic operations
Slower execution	Faster execution
Simple implementation	More efficient implementation

Conclusion

Non-restoring division is preferred in modern processors because it reduces unnecessary operations and improves execution speed.

5. IEEE 754 Floating Point Representation

IEEE 754 standard is used for representing real numbers in computers.

Given Number

$$-13.25$$

Binary Conversion

Integer part:

$$13 = 1101_2$$

Fractional part:

$$0.25 = 0.01_2$$

Therefore:

$$13.25 = 1101.01_2$$

Normalized form:

$$1.10101 \times 2^3$$

Single Precision Representation (32-bit)

Field	Size
Sign Bit	1 bit
Exponent	8 bits
Mantissa	23 bits

Sign Bit

Since the number is negative:

$$Sign = 1$$

Exponent

Bias value = 127

Actual exponent = 3

$$Exponent = 3 + 127 = 130$$

Binary:

$$130 = 10000010$$

Mantissa

Mantissa:

$$101010000000000000000000$$

Double Precision Representation (64-bit)

Field	Size
Sign Bit	1 bit
Exponent	11 bits
Mantissa	52 bits

Exponent bias = 1023

$$Exponent = 3 + 1023 = 1026$$

Binary: $1026 = 10000000010$

Mantissa:

10101000

Floating Point Addition Process

Floating-point addition is performed in the following steps:

1. Align Exponents

The smaller exponent is adjusted to match the larger exponent.

2. Add Mantissas

The significands are added.

3. Normalize Result

The result is converted into standard scientific notation.

4. Rounding

Extra bits are removed according to IEEE rounding rules.

5. Store Result: he final sign, exponent, and mantissa are stored.

Precision Problems

Floating-point operations may cause:

- Loss of precision.
- Rounding errors.
- Overflow and underflow.

Double precision provides higher accuracy because it uses a larger exponent and mantissa field.

Conclusion

Computer arithmetic operations use different techniques such as 2's complement arithmetic, Carry Look-Ahead Adders, Booth multiplication, division algorithms, and IEEE 754 floating-point representation to improve accuracy and performance. These methods are essential components of modern processor design.